

Task 1.1

```
# Abstract class should be derived from ABC class present in the abc module
# "" "" contains both abstract and non abstract method

from abc import *

class Person (ABC):
    #Constructor
    def __init__(self,name,address):
        self.__name= name
        self.__address= address
    #Setters
    def setName(self, name):
        self.__name=name

    def setAddress(self,address):
        self.__address=address
    #Getters
    def getName(self):
        return self.__name

    def getAddress(self):
        return self.__address
    #Display
    def __str__(self):
        return f"Person Name: {self.getName()} and Address is: {self.getAddress()}"

    #Abstract method
    @abstractmethod
    def studentship(self):
        pass

class Student(Person):
    #Constructor
    def __init__(self, name, address,studentID):
        super().__init__(name, address)
        self.__studentID= studentID
    #Checking if the person is student or not
    def studentship(self):
        if self.__studentID==0:
            print("This person is Not a Student")
```

```

        else:
            print("This person is a student")

# method overriding
def __str__(self):
    return super().__str__()

stud1= Student("Peter Miller", "Frenchay Campus",0)
stud2= Student("Max Miller", "Glenside Campus",1234)

stud1.studentship()
stud2.studentship()
print(stud1)
print(stud2)

```

Output:

```

nts\Practical10\student.py"
This person is Not a Student
This person is a student
Person Name: Peter Miller and Address is: Frenchay Campus
Person Name: Max Miller and Address is: Glenside Campus
PS C:\TBC\Sem2\POP_Project>

```

Task 1.2

```
# Abstract class should be derived from ABC class present in the abc module
# "" "" contains both abstract and non abstract method

from abc import *

class Person (ABC):
    #Constructor
    def __init__(self,name,address):
        self.__name= name
        self.__address= address
    #Setters
    def setName(self, name):
        self.__name=name

    def setAddress(self,address):
        self.__address=address
    #Getters
    def getName(self):
        return self.__name

    def getAddress(self):
        return self.__address
    #Display
    def __str__(self):
        return f"Person Name: {self.getName()} and Address is: {self.getAddress()}"

    #Abstract method
    @abstractmethod
    def studentship(self):
        pass

class Student(Person):
    #Constructor
    def __init__(self, name, address,studentID):
        super().__init__(name, address)
        self.__studentID= studentID
    #Checking if the person is student or not
    def studentship(self):
        if self.__studentID==0:
            print("This person is Not a Student")
```

```

        else:
            print("This person is a student")

# method overriding
def __str__(self):
    return super().__str__()

stud1= Student("Peter Miller", "Frenchay Campus",0)
stud2= Student("Max Miller", "Glenside Campus",1234)

stud1.studentship()
print(stud1)
stud2.studentship()
print(stud2)

p = Person("Abstract Person","Heaven")

"""
An abstract class Person can't be instantiated (object creation is not possible).
Must be subclass which inherit abstract class and implement the abstract method.
"""

```

Output:

```

Person Name: Max Miller and Address is: Glenside Campus
Traceback (most recent call last):
  File "c:\TBC\Sem2\POP_Project\Weekly Assignments\Practical10\student.py", line 58, in <module>
    p = Person("Abstract Person","Heaven")
TypeError: Can't instantiate abstract class Person without an implementation for abstract method 'studentship'
PS C:\TBC\Sem2\POP_Project> 

```

Task 1.3

```
from abc import *

class Animal(ABC):
    def __init__(self, numberOfLegs):
        self.__numberOfLegs = numberOfLegs
    #setter
    def setNumLegs(self, numberOfLegs):
        self.__numberOfLegs = numberOfLegs
    #getter
    def getNumLegs(self):
        return self.__numberOfLegs

    #abstract methods
    @abstractmethod
    def eat(self):
        pass
    @abstractmethod
    def walk(self):
        pass

class Pet(ABC):
    def __init__(self, name):
        self.__name = name.capitalize()

    def setName(self, name):
        self.__name = name.capitalize()

    def getName(self):
        return self.__name

    @abstractmethod
    def play(self):
        pass

class Cat(Animal, Pet):
    def __init__(self, name, numberOfLegs):
        #To call both constructor we need to specify constructor to call.
        #otherwise it'll call the 1st parameterised constructor which is Animal
class
        # When parent class is called directly, we must pass self to know the
current object(instance)
        Animal.__init__(self, numberOfLegs)
        Pet.__init__(self, name)
```

```

#Abstract method of base class
def eat(self):
    print(f"{self.getName()} eats mice")

def walk(self):
    print(f"{self.getName()} walks with {self.getNumLegs()} legs")

def play(self):
    print(f"{self.getName()} plays with tail")

#Multiple inheritance
class Fish(Animal,Pet):
    def __init__(self, name, numberOfLegs):
        Animal.__init__(self,numberOfLegs)
        Pet.__init__(self,name)

    #Abstract method of base class
    def eat(self):
        print(f"{self.getName()} eats plants")

    def walk(self):
        print(f"{self.getName()} can't walk")

    def play(self):
        print(f"{self.getName()} plays pokemon")

class Spider(Animal):
    def __init__(self, name="Spider", numberOfLegs=8):
        #No need for self cause super funtion
        # automatically passes the current instance(object)
        super().__init__(numberOfLegs)
        self.name = name

    #Abstract method of base class
    def eat(self):
        print(f"{self.name} eats fly")

    def walk(self):
        print(f"{self.name} walks with {self.getNumLegs()} legs")

s= Spider()
c= Cat("nini",4)
f= Fish("Nemo",0)

```

```
s.eat()  
s.walk()  
c.eat()  
c.walk()  
c.play()  
f.eat()  
f.walk()  
f.play()
```

Output:

```
Nemo plays pokemon  
PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\POP_Project\Weekly  
nts\Practical10\animal_class.py"  
  
Spider eats fly  
Spider walks with 8 legs  
Nini eats mice  
Nini walks with 4 legs  
Nini plays with tail  
Nemo eats plants  
Nemo cant walk  
Nemo plays pokemon  
PS C:\TBC\Sem2\POP_Project> 
```

```

from abc import *

class Animal(ABC):
    def __init__(self, numberOfLegs):
        self.__numberOfLegs = numberOfLegs
    #setter
    def setNumLegs(self, numberOfLegs):
        self.__numberOfLegs = numberOfLegs
    #getter
    def getNumLegs(self):
        return self.__numberOfLegs

    #abstract methods
    @abstractmethod
    def eat(self):
        pass
    @abstractmethod
    def walk(self):
        pass

class Pet(ABC):
    def __init__(self, name):
        self.__name = name.capitalize()

    def setName(self, name):
        self.__name = name.capitalize()

    def getName(self):
        return self.__name

    @abstractmethod
    def play(self):
        pass

class Cat(Animal, Pet):
    def __init__(self, name, numberOfLegs):
        #To call both constructor we need to specify constructor to call.
        #otherwise it'll call the 1st parameterised constructor which is Animal
class
        # When parent class is called directly, we must pass self to know the
        current object(instance)
        Animal.__init__(self, numberOfLegs)
        Pet.__init__(self, name)
        #Abstract method of base class

```



```

def eat(self):
    print(f"{self.getName()} eats mice")

def walk(self):
    print(f"{self.getName()} walks with {self.getNumLegs()} legs")

def play(self):
    print(f"{self.getName()} plays with tail")

#Multiple inheritance
class Fish(Animal,Pet):
    def __init__(self, name, numberOfLegs):
        Animal.__init__(self,numberOfLegs)
        Pet.__init__(self,name)

    #Abstract method of base class
    def eat(self):
        print(f"{self.getName()} eats plants")

    def walk(self):
        print(f"{self.getName()} can't walk")

    def play(self):
        print(f"{self.getName()} plays pokemon")

class Spider(Animal):
    def __init__(self, name="Spider", numberOfLegs=8):
        #No need for self cause super funtion
        # automatically passes the current instance(object)
        super().__init__(numberOfLegs)
        self.name = name

    #Abstract method of base class
    def eat(self):
        print(f"{self.name} eats fly")

    def walk(self):
        print(f"{self.name} walks with {self.getNumLegs()} legs")

s= Spider()
c= Cat("Max",4)
f= Fish("Nemo",0)
s.eat()

```

```
s.walk()

#Polymorphism
for x in (c,f):
    x.eat()
    x.walk()
    x.play()
```

Output:

```
PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\PO
nts\Practical10\animal_class.py"
Spider eats fly
Spider walks with 8 legs
Max eats mice
Max walks with 4 legs
Max plays with tail
Nemo eats plants
Nemo can't walk
Nemo plays pokemon
PS C:\TBC\Sem2\POP_Project> █
```

Task 2.1

```
def input_numbers():  
  
    a = float(input("Enter first number: "))  
    b = float(input("Enter second number: "))  
  
    try:  
        print(f"{a}/{b} is {a/b}")  
  
    except ZeroDivisionError:  
        print("\nCannot Divide by zero\n")  
        input_numbers()  
  
input_numbers()
```

Output:

```
nts\Practical10\zeroDivisionError.py  
Enter first number: 10  
Enter second number: 20  
10.0/20.0 is 0.5  
PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\POP_  
● nts\Practical10\zeroDivisionError.py"  
Enter first number: 10  
Enter second number: 0  
  
Cannot Divide by zero  
  
Enter first number: 10  
Enter second number: 0  
  
Cannot Divide by zero  
  
Enter first number: 0  
Enter second number: 10  
0.0/10.0 is 0.0  
● PS C:\TBC\Sem2\POP_Project> |
```

Task 2.2

```
def input_numbers():  
  
    a = float(input("Enter first number: "))  
    b = float(input("Enter second number: "))  
  
    try:  
        print(f"{a}/{b} is {a/b}")  
  
    except ZeroDivisionError as exp:  
        print("\n")  
        print(exp)  
        print("Cannot Divide by zero\n")  
        input_numbers()  
  
input_numbers()
```

Output:

```
Project\Weekly Assignments\Practical10\zeroDivisionError.py  
● PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\Projects\Practical10\zeroDivisionError.py"  
Enter first number: 10  
Enter second number: 0  
  
float division by zero  
Cannot Divide by zero  
  
Enter first number: 10  
Enter second number: 2  
10.0/2.0 is 5.0  
○ PS C:\TBC\Sem2\POP_Project> █
```

Task 2.3

```
#Value Error

def read_mark():
    mark = int(input("Enter your POP In-class tests mark: "))

    if mark<0 or mark>500:
        raise ValueError ("Invalid Mark")

    return mark

try:
    val = read_mark()
    print(f"Your mark is {val}")

except ValueError as e:
    print(e)
```

Output:

```
nts\Practical10\valueError.py"
Enter your POP In-class tests mark: 450
Your mark is 450
● PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\POP_Proje
nts\Practical10\valueError.py"
Enter your POP In-class tests mark: -450
Invalid Mark
○ PS C:\TBC\Sem2\POP_Project> █
```

```
PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\POP_Proje
nts\Practical10\valueError.py"
Your mark is 450
PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\POP_Proje
nts\Practical10\valueError.py"
Enter your POP In-class tests mark: -450
Invalid Mark
PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\POP_Proje
nts\Practical10\valueError.py"
Enter your POP In-class tests mark: dfd
invalid literal for int() with base 10: 'dfd'
PS C:\TBC\Sem2\POP_Project> █
```

Task 2.4

```
#Index error: Error which occurs when we try to access non exist index in the
list (index out of range of that list)

try:
    a=['a','b','c']
    i = int(input("Enter the index of the element you want to print: "))
    print(a[i])
except LookupError:
    print("Index Error Exception: List index out of range!!!")
else:
    print("No error!")
```

Output:

```
PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\POP_F
● nts\Practical10\index_Error.py"
Enter the index of the element you want to print: 4
Index Error Exception: List index out of range!!!
PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\POP_F
● nts\Practical10\index_Error.py"
Enter the index of the element you want to print: 0
a
No error!
○ PS C:\TBC\Sem2\POP_Project> █
```

Task 2.5

```
class Loan:
    def __init__(self, annualInterestRate=2.5, numberOfYears=1, loanAmount=1000):
        self.__annualInterestRate = annualInterestRate
        self.__numberOfYears = numberOfYears
        self.__loanAmount = loanAmount

    #Setters
    def setAnnualInterestRate(self, annualInterestRate):
        self.__annualInterestRate = annualInterestRate

    def setNumberOfYears(self, numberOfYears):
        self.__numberOfYears = numberOfYears

    def setLoanAmount(self, loanAmount):
        self.__loanAmount = loanAmount

    #getters
    def getAnnualInterestRate(self):

        return self.__annualInterestRate

    def getNumberOfYears(self):

        return self.__numberOfYears

    def getLoanAmount(self):

        return self.__loanAmount

    def getMonthlyPayment(self):
        monthlyInterestRate = self.__annualInterestRate / 1200
        numberOfPayments = self.__numberOfYears*12
        # " \ " is a continuation character which tells python statement
continues on next line
        monthlyPayment= self.__loanAmount * monthlyInterestRate * (1 +
monthlyInterestRate)* numberOfPayments \
        / ((1+monthlyInterestRate)*numberOfPayments)

        return monthlyPayment

    def getTotalPayment(self):
        totalPayment= self.getMonthlyPayment() * self.__numberOfYears* 12
        return totalPayment
```

```
try:
    l = Loan(7.5,30,100000)
    print("Total payment: "+ str(l.getTotalPayment()))

    m= Loan(-1,3,3)
    print("Total payment: "+ str(m.getTotalPayment()))

except ValueError as e:
    print(e)
```

Output:

```
PS C:\TBC\Sem2\POP_Project> python -u "
nts\Practical10\loan.py"
Total payment: 224999.99999999994
Total payment: -0.08999999999999998
PS C:\TBC\Sem2\POP Project> |
```


Task 2.6

```
class Loan:
    def __init__(self, annualInterestRate=2.5, numberOfYears=1, loanAmount=1000):
        self.__annualInterestRate = annualInterestRate
        self.__numberOfYears = numberOfYears
        self.__loanAmount = loanAmount

        if self.__annualInterestRate <= 0:
            raise ValueError("Invalid Annual Interest Rate")
        if self.__numberOfYears <= 0:
            raise ValueError("Invalid number of years")
        if self.__loanAmount <= 0:
            raise ValueError("Invalid Loan Amount")

    #Setters
    def setAnnualInterestRate(self, annualInterestRate):
        self.__annualInterestRate = annualInterestRate

    def setNumberOfYears(self, numberOfYears):
        self.__numberOfYears = numberOfYears

    def setLoanAmount(self, loanAmount):
        self.__loanAmount = loanAmount

    #getters
    def getAnnualInterestRate(self):
        if self.__annualInterestRate <= 0:
            raise ValueError("Invalid Annual Interest Rate")
        return self.__annualInterestRate

    def getNumberOfYears(self):
        if self.__numberOfYears <= 0:
            raise ValueError("Invalid number of years")
        return self.__numberOfYears

    def getLoanAmount(self):
        if self.__loanAmount <= 0:
            raise ValueError("Invalid Loan Amount")
        return self.__loanAmount

    def getMonthlyPayment(self):
```

```

        monthlyInterestRate = self.__annualInterestRate / 1200
        numberOfPayments = self.__numberOfYears*12
        # " \ " is a continuation character which tells python statement
continues on next line
        monthlyPayment= self.__loanAmount * monthlyInterestRate * (1 +
monthlyInterestRate)* numberOfPayments \
        / ((1+monthlyInterestRate)*numberOfPayments)

        return monthlyPayment

    def getTotalPayment(self):
        totalPayment= self.getMonthlyPayment() * self.__numberOfYears* 12
        return totalPayment

try:
    l = Loan(7.5,30,100000)
    print("Total payment: "+ str(l.getTotalPayment()))

    m= Loan(-1,3,3)
    print("Total payment: "+ str(m.getTotalPayment()))

except ValueError as e:
    print(e)

```

Output:

```

nts\Practical10\loan.py"
Total payment: 224999.999999999994
Invalid Annual Interest Rate
PS C:\TBC\Sem2\POP_Project> 

```

Task 3.1

```
#File handling
f = open("Weekly Assignments/Practical10/myFile.txt","r")
print(f.read())
f.close()
```

Output:

```
PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\POP_Project\Weekly Assignments\Practical10\myFile.py"
The Object-Oriented Design Principles are the core of OOP programming.
It's essential to learn the basics of Object-oriented programming.
Like Abstraction, Encapsulation, Polymorphism, and Inheritance.
PS C:\TBC\Sem2\POP_Project> █
```

Task 3.2

```
#File handling
f = open("Weekly Assignments/Practical10/myFile.txt","r")
print(f.read())
f.close()
print()
f= open("Weekly Assignments/Practical10/yourFile.txt","r")
print(f.readline())
f.close()
```

Output:

```
PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\POP_Project\Weekly Assignm
nts\Practical10\myFile.py"
The Object-Oriented Design Principles are the core of OOP programming.
It's essential to learn the basics of Object-oriented programming.
Like Abstraction, Encapsulation, Polymorphism, and Inheritance.

The Object-Oriented Design Principles are the core of OOP programming.
PS C:\TBC\Sem2\POP Project> █
```

Task 3.3

```
#File handling
f = open("Weekly Assignments/Practical10/myFile.txt","r")
print(f.read())
f.close()
print()
f= open("Weekly Assignments/Practical10/yourFile.txt","r")
print(f.readline())
print(f.readline())
f.close()
```

Output:

```
PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\POP_Project\Weekly Assignments\Practical10\myFile.py"
The Object-Oriented Design Principles are the core of OOP programming.
It's essential to learn the basics of Object-oriented programming.
Like Abstraction, Encapsulation, Polymorphism, and Inheritance.

The Object-Oriented Design Principles are the core of OOP programming.

It's essential to learn the basics of Object-oriented programming.

PS C:\TBC\Sem2\POP_Project> █
```

Task 3.4

```
#File handling
f = open("Weekly Assignments/Practical10/myFile.txt","r")
print(f.read())
f.close()
print()
f= open("Weekly Assignments/Practical10/yourFile.txt","r")
print(f.readline())
print(f.readline())
f.close()

try:
    f=open("Weekly Assignments/Practical10/yourFile1.txt","r")
    print(f.readline())
    f.close()

except FileNotFoundError as e:
    print(e)
```

Output:

```
PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\POP_Project\Weekly Assignme
nts\Practical10\myFile.py"
It's essential to learn the basics of Object-oriented programming.

PS C:\TBC\Sem2\POP_Project> python -u "c:\TBC\Sem2\POP_Project\Weekly Assignme
● nts\Practical10\myFile.py"
The Object-Oriented Design Principles are the core of OOP programming.
It's essential to learn the basics of Object-oriented programming.
Like Abstraction, Encapsulation, Polymorphism, and Inheritance.

The Object-Oriented Design Principles are the core of OOP programming.

It's essential to learn the basics of Object-oriented programming.

[Errno 2] No such file or directory: 'Weekly Assignments/Practical10/yourFile1
.txt'
○ PS C:\TBC\Sem2\POP_Project> █
```

main* 0 0 0 Spaces: 4 UTF-8 CRLF { } Python Python 3.13 (64-bit)